

COMP1549: Advanced Programming

Secure Programs

Dr Taimoor Khan

WWW

- [Personal](#)
- Operational Lead - NCSC accredited [ACE-CSR](#)
- Deputy Director - NCSC accredited [ACE-CSE](#)
- [RITICS Fellow](#) – Imperial College London

12th February 2026 - Week 5

Secure Programs

A program is a set of instructions that encode requirements (or intentions) that involve

- concepts from real-life, e.g., age, and distance

A program is **secure** iff it is **not vulnerable**

Vulnerability is

- Either a weakness in a program
- Or a feature of a program that can be misused

which can be exploited by an adversary for harm

How to develop a vulnerability free program?

- By encoding concepts in an adequate way, i.e., without missing key details that may be misinterpreted

But current programming languages

- Either do not have built-in support for the concepts, e.g., age
- Or partially supports the concepts subject to validity, e.g., time

How to encode concepts adequately in a program?

- Through **describing/modelling** key details of the concepts to the program or run-time

We focus on vulnerabilities that arise from programming languages

One way to model simple but key details of the concepts is

- Using Java annotations
 - To avoid **compile-time errors** but
 - Can also be used to avoid run-time errors

Checker Framework (1)

- A Java compiler plug-in framework that extends type checking to catch bugs at compile time
 - Extends data types with additional constraints
 - ❖ Types represent values that form a concept
 - ❖ Type constraints introduces legit use of operations with the values
- The checker guarantees that
 - type annotations reflect facts about run-time values, and
 - vulnerable operations (on values) are not performed

Checker Framework (2)

- The checker is open-source and free, accessible via <https://checkerframework.org>
- The checker guarantees that
 - type annotations reflect facts about run-time values, and
 - vulnerable operations (on values) are not performed

Example – Null values

- Write a program to get name as an input
 - Where first and last name cannot be null, while
 - Middle name can be null
- Can a Java compiler find any error in this program?

```
public void getName(String firstName, String middleName, String lastName){  
    System.out.println("First name is "+firstName.toString());  
    System.out.println("Middle name is "+middleName.toString());  
    System.out.println("Last name is "+lastName.toString());  
  
}
```

Demo

```
public void getName(String firstName, @Nullable String middleName, String lastName){  
    System.out.println("First name is "+firstName.toString());  
    System.out.println("Middle name is "+middleName.toString());  
    System.out.println("Last name is "+lastName.toString());  
}
```

<http://tinyurl.com/yeyrcrux>

@Nullable

Package org.checkerframework.checker.nullness.qual

Annotation Interface Nullable

```
@Documented↗  
@Retention↗(RUNTIME↗)  
@Target↗({TYPE_USE↗, TYPE_PARAMETER↗})  
@SubtypeOf({})  
@QualifierForLiterals(NULL)  
@DefaultFor(types=java.lang.Void.class↗)  
public @interface Nullable
```

Example – Injection vulnerability

- Write a program to fetch data from a database using a
 - Query that requires input username from the user
- Can a Java compiler find any error in this program?

```
public String getUserInput() {  
    return "untrustedInput";  
}
```

```
public void processRequest() {  
    String input = getUserInput();  
    executeQuery(input);  
}
```

```
public void executeQuery(String input) {  
    // Run the SQL Query with the provided input  
}
```

Example – Vulnerable run

What if an attacker inputs `username` as

1. `admin';-`

- ▶ `select * from TBL_USERS where username = 'admin';-and password = ' '`
- ▶ shows all credentials of the user `admin`

2. `'; exec master..xp_cmdshell 'net users username password /add';-`

- ▶ `select * from TBL_USERS where username = '' ; exec master..xp_cmdshell 'net users username password /add';-and password = ' '`
- ▶ creates a new Windows user

3. `'; exec master..xp_cmdshell 'net localgroup Administrators username /add';-`

- ▶ `select * from TBL_USERS where username = '' ; exec master..xp_cmdshell 'net localgroup Administrators username /add';-and password = ' '`
- ▶ assigns administrative rights to the new user

Demo

```
public String getUserInput() {  
    return "untrustedInput";  
}
```

```
public void processRequest() {  
    @Tainted String input = getUserInput();  
    executeQuery(input);  
}
```

```
public void executeQuery(@Untainted String input) {  
    // Run the SQL Query with the provided input  
}
```

<http://tinyurl.com/5n76ntsw>

@Tainted

Package `org.checkerframework.checker.tainting.qual`

Annotation Interface Tainted

```
@Documented↗  
@Retention↗(RUNTIME↗)  
@Target↗({TYPE_USE↗, TYPE_PARAMETER↗})  
@SubtypeOf({})  
@DefaultQualifierInHierarchy  
public @interface Tainted
```

Denotes a possibly-tainted value: at run time, the value might be tainted or might be untainted.

See Also:

Untainted, TaintingChecker

See the Checker Framework Manual:

Tainting Checker[↗]

More...

- The checker can be extended to
 - Introduce new types of your choice with desired constraints
 - Update existing types with additional constraints
 - Model advanced and generic concepts as types in an automated way
 - ...

Further demo

End of week 5!